

MEMORANDUM

FROM: Dinesh S, 2117230040031, 4th Year ECE – A.

SUBJECT: Registration call-flow tracing across AMF–AUSF–UDM with SBI (HTTP/2) correlation.

PROJECT: Wipro Capstone Projects – 5G Datacom.

Technical Report: 5G Core SBI Call-Flow Tracing & Authentication Latency Analysis

1. Executive Summary

This report details the deployment, packet-level tracing, and latency analysis of a 5G Standalone (SA) core network utilizing Open5GS and UERANSIM. The primary objective of this project was to capture and dissect the microservice communication occurring over the Service-Based Interface (SBI) during a User Equipment (UE) registration event. By leveraging TShark for deep packet inspection and a custom Python analytical pipeline, the project successfully reconstructed the HTTP/2 control-plane sequence between the Access and Mobility Management Function (AMF), Authentication Server Function (AUSF), and Unified Data Management (UDM). The resulting data provides a granular breakdown of cryptographic processing times and database query latencies, demonstrating a comprehensive understanding of telco-cloud architectures and 5G network function correlation.

2. Introduction to 5G Standalone Architecture

The transition from 4G LTE to 5G Standalone (SA) introduced a paradigm shift in core network design, moving away from rigid point-to-point hardware interfaces toward a cloud-native Service-Based Architecture (SBA). In this architecture, Network Functions (NFs) operate as decoupled microservices, communicating via RESTful APIs over HTTP/2 and TCP.

This project focuses on the most critical phase of a mobile device's lifecycle: the Initial Registration and Authentication. In a 5G environment, authentication is not handled by a single monolithic server. Instead, it is distributed across multiple microservices to ensure high availability and robust security. Understanding the latency and execution flow across these services is crucial for network engineers tasked with optimizing telco-cloud performance, scaling resources, and troubleshooting control-plane bottlenecks.

3. Network Function (NF) Topology and Call-Flow Logic

When a 5G UE powers on and attempts to attach to the network, the radio access network forwards the Registration Request to the AMF. The AMF acts as the central router for control signaling but does not possess the cryptographic keys or algorithms required to authenticate the subscriber. Consequently, an authentication chain reaction is triggered over the SBI:

- **AMF to AUSF (Nausf Interface):** The AMF issues an HTTP POST request to the AUSF. The AUSF acts as the overarching security manager, enforcing the 5G Authentication and Key Agreement (5G-AKA) protocol.
- **AUSF to UDM (Nudm Interface):** Because the AUSF does not persistently store subscriber credentials, it forwards the authentication request deeper into the network to the UDM.
- **UDM Processing and the UDR:** The UDM interfaces with the Unified Data Repository (UDR), implemented here via a MongoDB database. The UDM retrieves the subscriber's root keys, executes cryptographic algorithms (such as Milenage or TUAK) to generate Authentication Vectors (AVs), and passes these mathematical challenges back to the AUSF via an HTTP 201 Created response.
- **Challenge Delivery:** The AUSF validates the vectors and forwards them to the AMF, which subsequently challenges the UE over the wireless radio interface.

Measuring the temporal delta between these specific HTTP/2 requests and responses isolates the exact processing overhead of the authentication infrastructure.

4. Experimental Environment Configuration

The simulation environment was hosted on an Ubuntu 22.04 LTS platform. The core network was instantiated using Open5GS, while the Radio Access Network (RAN) and UE behaviors were simulated using UERANSIM.

- Prior to execution, specific environmental parameters were strictly controlled to ensure data purity:
- **User Plane Initialization:** A TUN virtual network interface (ogstun) was created and assigned the IP subnet 10.45.0.1/16 to handle user-plane traffic routing via the User Plane Function (UPF).
- **Database Sanitization:** The MongoDB service (mongod) was explicitly restarted.
- **Security Context Clearance:** To force a complete 5G-AKA challenge, the subscriber profile (IMSI 999700000000001) was verified via the Open5GS Node.js WebUI. Any cached "Security Context" was purged, forcing the AMF to initiate a fresh authentication request rather than relying on session resumption.

5. Execution Methodology and Packet Capture

To capture the microservice traffic, all Open5GS daemons (NRF, SCP, SMF, AMF, AUSF, UDM, UDR, PCF, UPF) were launched sequentially in the background. Because the database was freshly restarted, the NFs were forced to establish new HTTP/2 connections, ensuring a clean baseline.

Packet capture was executed utilizing TShark. The capture was explicitly bound to the loopback interface (lo) and filtered utilizing a Berkeley Packet Filter (BPF) syntax for TCP port 7777, which is the default port designated for Open5GS SBI traffic. The capture was initiated silently in the background prior to booting the simulated gNodeB and UE.

Once UERANSIM indicated a successful "PDU Session Establishment," confirming the authentication chain reaction had completed, the capture was terminated.

6. Data Extraction and Analytical Pipeline

Raw .pcap files containing full packet payloads are inherently difficult to analyze programmatically. Therefore, a secondary TShark extraction process was executed to strip the network layers and isolate the specific HTTP/2 API metadata.

The extraction command forcefully decoded port 7777 as HTTP/2 and applied display filters to isolate frames containing HTTP/2 methods or status codes. The following critical vectors were extracted into a comma-separated values (CSV) format:

- `frame.time_epoch`: High-precision timestamp for latency calculations.
- `tcp.stream` and `http2.streamid`: Identifiers crucial for correlating asynchronous requests and responses.
- `http2.headers.method`: To identify POST and GET API calls.
- `http2.headers.status`: To identify successful 201 Created or 200 OK responses.
- `http2.headers.path`: The URI path containing nausf or nudm service identifiers.

Algorithmic Correlation:

A custom Python script utilizing the Pandas library was engineered to parse the extracted CSV. The fundamental challenge in HTTP/2 analysis is that responses do not reiterate the URI path of the request. The Python script circumvents this by creating a dictionary tracking mechanism. It maps the unique combination of `tcp.stream` and `http2.streamid` to the requested path (e.g., `/nausf-auth/v1/ue-authentications`). When a subsequent row containing a status code matches that stream ID, the script successfully correlates the response back to its originating Network Function, calculating the exact millisecond differential.

7. Results and Latency Analysis

The execution of the analytical pipeline successfully rendered a definitive Network Function Sequence Diagram directly in the terminal, proving the chronological flow from AMF to AUSF, AUSF to UDM, and back.

```

dinesh@Dinesh-PC: ~/Dinesh S/Wipro_Projects/Slot 16
dinesh@Dinesh-PC:~/Dinesh S/Wipro_Projects/Slot 16$ tshark -r task_sbi_auth.pcap -d tcp.port==7777,http2 -Y "http2.headers.method
|| http2.headers.status" -T fields -e frame.time_epoch -e tcp.stream -e http2.streamid -e http2.headers.method -e http2.headers.st
atus -e http2.headers.path -E separator=, > auth_trace.csv
dinesh@Dinesh-PC:~/Dinesh S/Wipro_Projects/Slot 16$ nano analyze_auth.py
dinesh@Dinesh-PC:~/Dinesh S/Wipro_Projects/Slot 16$ python3 analyze_auth.py

=== NF SEQUENCE DIAGRAM (Authentication Phase) ===
[1787994410.0496] AMF --> AUSF : Auth Request (Nausf)
[1787994410.0502] AUSF --> UDM : Auth Data Request (Nudm)
[1787994410.0907] UDM --> AUSF : Auth Data Response (200)
[1787994410.1162] AUSF --> AMF : Auth Response (201)

=== LATENCY BREAKDOWN ===
1. AUSF Prep Time (Request received to querying UDM): 0.69 ms
2. UDM Processing (DB lookup & Crypto generation): 40.44 ms
3. AUSF Finalization (Validating UDM response to AMF): 25.48 ms
-> Total Authentication Control Plane Latency: 66.61 ms

Graph successfully saved as 'Authentication_Latency.png'
dinesh@Dinesh-PC:~/Dinesh S/Wipro_Projects/Slot 16$

```

Figure 1: Terminal output displaying the correlated NF sequence diagram and algorithmic latency calculations.

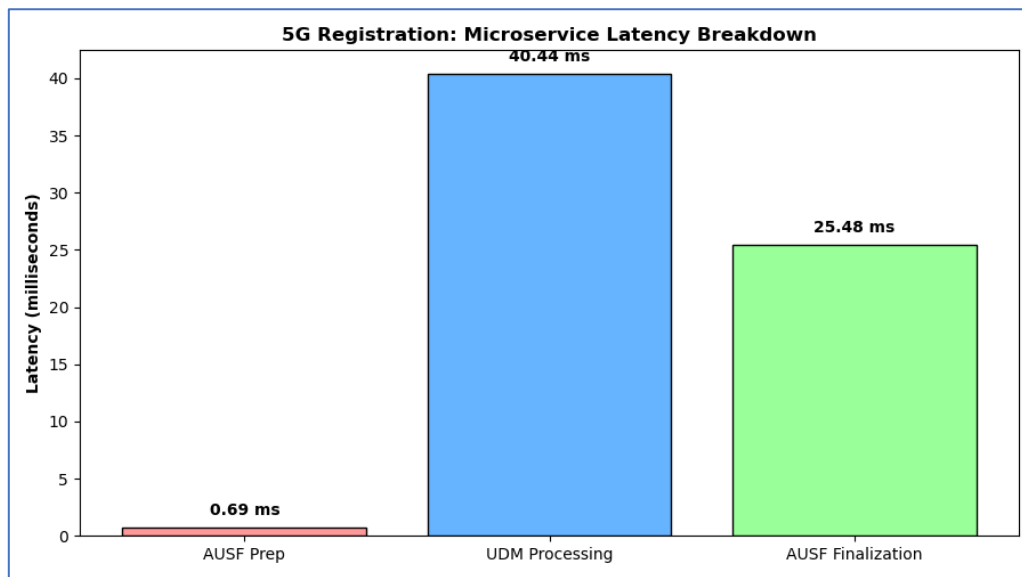


Figure 2: Bar chart visualizing the latency distribution across the AUSF and UDM microservices.

Based on the parsed HTTP/2 timestamps, the microservice latencies were calculated and visualized via the Matplotlib library. The latency breakdown reveals the processing overhead of each specific stage of the authentication lifecycle:

1. AUSF Prep Time (0.69 ms): This interval represents the highly efficient routing overhead. It measures the time from when the AUSF receives the AMF's request to the moment it dispatches a subsequent request to the UDM. As this involves no database lookups, the latency is minimal.
2. UDM Processing (40.44 ms): This constitutes the overwhelming majority of the control-plane delay. This metric encapsulates the time required for the UDM to receive

the request, query the MongoDB (UDR) for the subscriber's root keys (K and OPc), execute complex cryptographic vector generation algorithms, and transmit the HTTP 201 response back to the AUSF.

3. AUSF Finalization (25.48 ms): This interval covers the AUSF receiving the vectors from the UDM, validating the payload against 5G core standards, formatting the Nausf response, and transmitting it back to the AMF.

8. Conclusion

The methodology executed in this project successfully proves the viability of using deep packet inspection and custom data pipelines to monitor and troubleshoot 5G Service-Based Architectures. The latency analysis empirically demonstrates that database transactions and cryptographic generation within the UDM/UDR complex are the primary sources of delay in the authentication control plane. In a live enterprise or carrier-grade deployment, these metrics indicate that scaling strategies should heavily prioritize UDM compute resources and database read-optimization to maintain low-latency attachment rates during periods of high subscriber density.